Lab 5: Static Code Analysis

Pylint · Bandit · Flake8 on auction_system.py (Online Auction and Bidding System)

Name	Manasa K Yadav
SRN	PES1UG25CS824
Course	Software Engineering – Lab 5
Assigned repo	github.com/SETAPESU26/L5_65_auction-bidding
Submission repo	github.com/ManasaKYadav/SE-Labs-PES1UG25CS824 (folder LAB5)
Date	30 October 2026

Tool	Before	After
Pylint score	6.09 / 10 (33 messages)	10.00 / 10 (0 messages)
Bandit issues	3 (2 High, 1 Medium)	0 – No issues identified
Flake8 warnings	19	0

1. Introduction to Static Code Analysis

Static code analysis examines source code **without executing it** to find potential bugs, security vulnerabilities and style violations early in development, when they are cheapest to fix. Analysers parse the code and check it against rules for correctness, security and coding standards. In this lab three industry-standard Python tools were used:

Tool	Role	What it checks
Pylint	The strict code reviewer	Code quality and logical problems: naming conventions (PEP 8 snake_case), missing docstrings, broad exceptions, attributes defined outside __init__, unnecessary else, import order, design smells. Gives a score out of 10.
Flake8	The grammar checker	PEP 8 style and formatting: blank lines between definitions, line length (79 chars), whitespace, trailing newlines, unused imports and syntax errors (pycodestyle + pyflakes).
Bandit	The app's security guard	Security vulnerabilities: insecure modules/imports, injection and XSS risks, unsafe template settings, hard-coded secrets. Reports each issue with a severity and confidence (Low / Medium / High).

1.1 Program under analysis

`auction_system.py` is an online auction and bidding system. An `Auction` class tracks reserve prices and closing times; the program accepts bids, picks the winner and final price (with a 5% buyer's premium), lists top bidders, renders HTML listings and bidder badges, exports bids as JSON and calls an XML-RPC payment gateway to charge the winner.

1.2 Procedure

- Created a virtual environment and installed the tools: `python -m pip install pylint bandit flake8 jinja2 defusedxml`
- Ran the original program and read its output to spot wrong behaviour.
- Ran the three tools and saved the reports (pylint_report_before.txt, bandit_report_before.txt, flake8_report_before.txt).
- Documented the issues, fixed them in three commits (Bandit, then Pylint + logic bugs, then Flake8), and re-ran the tools to produce the _after reports.

```
(venv) PS D:\PES_5TH_SEM\SE\LAB\LAB_5> python auction_system.py
Asha bids 3000: True
Ben bids 3000 (same amount): True
Asha bids 3050 (+50): True
Seller bids on own item: True
Bid at 9:30 AM (before 18:00 close): False
Late bid (next day): False
Winner: seller_sam
Final price (reserve 5000): 4200.0
Top bidders: ['seller_sam', 'asha', 'ben']
Rs.0 bid on LOT3 (reserve 800): True
LOT3 winner: carl
No-bid winner: None
Listing HTML: <h2><script>alert('pwned')</script>Old Radio</h2><p>Curr
ent bid: Rs.0</p>
Badge: <span class='bidder'><img src=x onerror=alert(1)></span>
(venv) PS D:\PES_5TH_SEM\SE\LAB\LAB_5>
```

Figure 1 – Output of the original program. Equal and +50 bids, a seller bid and a Rs.0 bid are accepted, a valid 9:30 bid is rejected, the winner is chosen below the reserve, and the HTML output contains unescaped <script> / <img onerror> payloads.

2. Issues Identified Before Fixing

The tools were run on the original file. Pylint rated the code **6.09/10**, Bandit found **3 security issues (2 High, 1 Medium)** and Flake8 reported **19 style violations**.

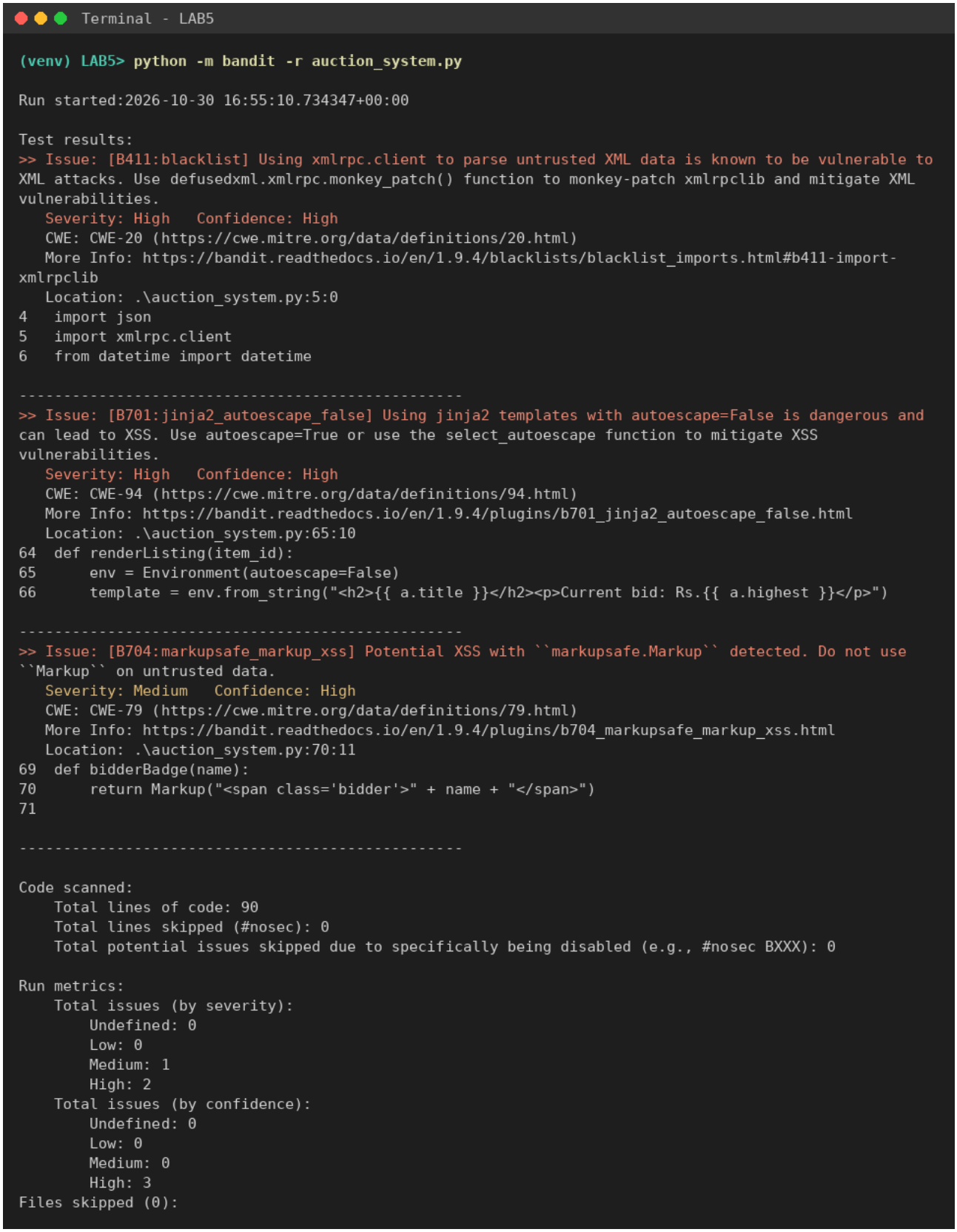


Figure 2 – Bandit before fixing: B411 (High), B701 (High), B704 (Medium).

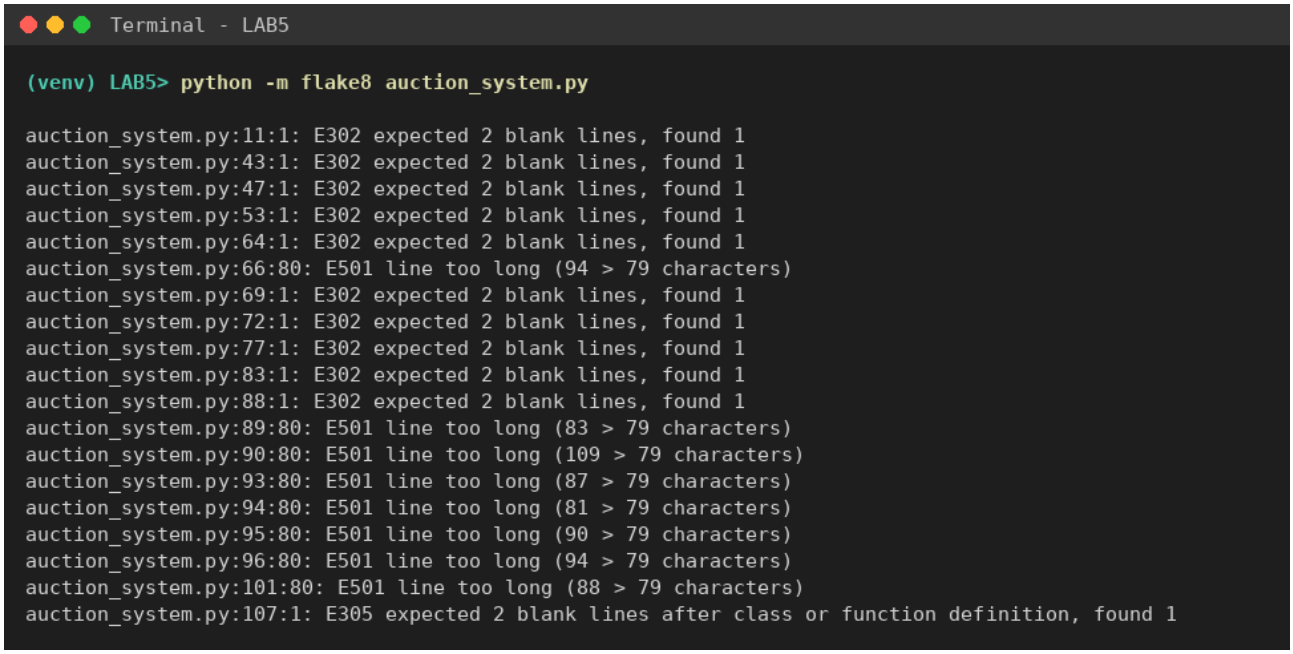


Figure 3 – Flake8 before fixing: E302 ×10, E501 ×8, E305 ×1.

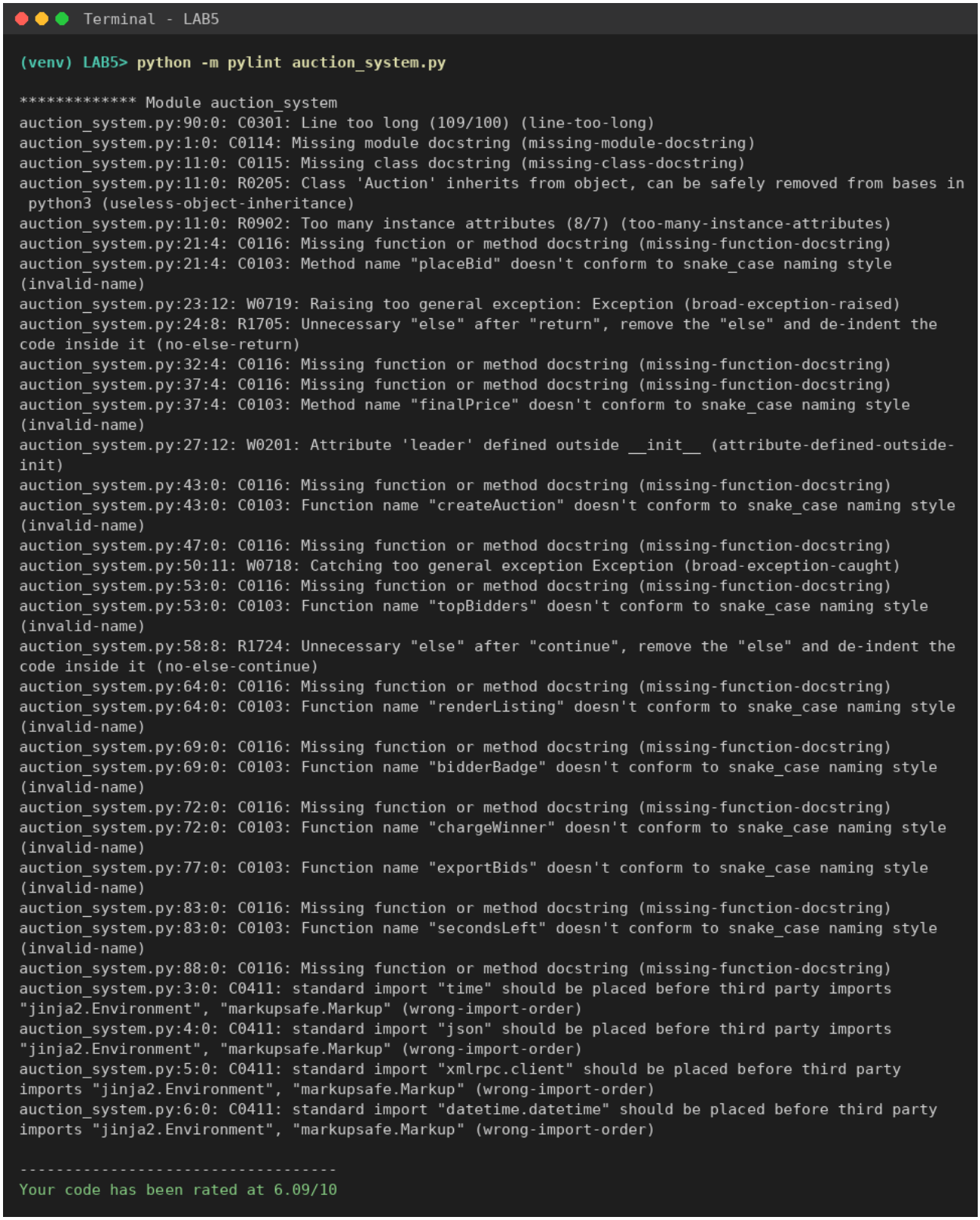


Figure 4 – Pylint before fixing: 33 messages, score 6.09/10.

3. Issue Identification Table

High and medium severity issues are listed first. Rows marked **Bug** are logic errors found by reading the program output – none of the three tools reported them.

Issue	Type	Line(s)	Description	Fix Approach
Insecure XML-RPC import (B411)	Security – High	5, 73	xmlrpc.client is vulnerable to XML attacks (e.g. entity expansion) when parsing untrusted responses	Import defusedxml.xmlrpc, call monkey_patch() and create the ServerProxy through defusedxml.xmlrpc.xmlrpc_client
Jinja2 autoescape disabled (B701)	Security – High	65	autoescape=False lets the <script> in an auction title reach the HTML unescaped (XSS)	Environment(autoescape=True)
Unsafe Markup (B704)	Security – Medium	70	Bidder name concatenated into Markup(), so attacker HTML is trusted (XSS)	Markup("<span ...>{}").format(name) – format() escapes the argument
String time comparison	Bug – High	22	Times compared as strings: "2026-12-01 9:30" > "2026-12-01 18:00", so a valid morning bid is rejected	Parse times with datetime.strptime and compare datetimes
Reserve price ignored	Bug – High	32–38	winner() and finalPrice() return a winner/price even when the highest bid is below the reserve	Return None if highest < reserve_price
Equal / too-small bids accepted	Bug – Medium	24	>= accepts equal bids; MIN_INCREMENT is defined but never used	Require amount >= highest + MIN_INCREMENT
Seller bids on own item	Bug – Medium	21–30	Allows shill bidding by the seller	Reject bids where bidder == seller
Rs.0 bid accepted	Bug – Medium	19, 24	highest starts at 0, so a 0 bid is accepted and wins	Reject amount <= 0
Broad exception raised / caught (W0719, W0718)	Pylint – Warning	23, 50	Generic Exception is raised and caught, hiding real errors such as a wrong item id	Custom AuctionClosedError; bid() catches only KeyError and AuctionClosedError
Attribute defined outside __init__ (W0201), too many attributes (R0902)	Pylint – Warning	11, 27	leader does not exist until the first bid; class had 8 attributes	highest and leader became @property values computed from bids
Naming, docstrings, imports (C0103, C0114–C0116, C0411, R0205, R1705, R1724)	Pylint – Convention	many	camelCase names, no docstrings, stdlib imports after third-party, class Auction(object), unnecessary else	snake_case names, docstrings everywhere, stdlib imports first, class Auction:, else removed
Blank lines / long lines (E302, E305, E501)	Flake8 – Style	many	Only 1 blank line between top-level definitions; lines over 79 characters	2 blank lines between definitions; long lines wrapped

4. Before and After Outputs of the Tools

Report file	Before fixing	After fixing
pylint_report_*.txt	33 messages – rated 6.09/10	No messages – rated 10.00/10
bandit_report_*.txt	3 issues: B411 High, B701 High, B704 Medium	No issues identified (0 High, 0 Medium, 0 Low)
flake8_report_*.txt	19 lines: E302 ×10, E501 ×8, E305 ×1	Empty – no violations

The complete contents of all six report files follow.

pylint_report_before.txt

```
************* Module auction_system
auction_system.py:90:0: C0301: Line too long (109/100) (line-too-long)
auction_system.py:1:0: C0114: Missing module docstring (missing-module-docstring)
auction_system.py:11:0: C0115: Missing class docstring (missing-class-docstring)
auction_system.py:11:0: R0205: Class 'Auction' inherits from object, can be safely removed from bases in python3 (
  useless-object-inheritance)
auction_system.py:11:0: R0902: Too many instance attributes (8/7) (too-many-instance-attributes)
auction_system.py:21:4: C0116: Missing function or method docstring (missing-function-docstring)
auction_system.py:21:4: C0103: Method name "placeBid" doesn't conform to snake_case naming style (invalid-name)
auction_system.py:23:12: W0719: Raising too general exception: Exception (broad-exception-raised)
auction_system.py:24:8: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the code inside it (
  no-else-return)
auction_system.py:32:4: C0116: Missing function or method docstring (missing-function-docstring)
auction_system.py:37:4: C0116: Missing function or method docstring (missing-function-docstring)
auction_system.py:37:4: C0103: Method name "finalPrice" doesn't conform to snake_case naming style (invalid-name)
auction_system.py:27:12: W0201: Attribute 'leader' defined outside __init__ (attribute-defined-outside-init)
auction_system.py:43:0: C0116: Missing function or method docstring (missing-function-docstring)
auction_system.py:43:0: C0103: Function name "createAuction" doesn't conform to snake_case naming style (invalid-name)
auction_system.py:47:0: C0116: Missing function or method docstring (missing-function-docstring)
auction_system.py:50:11: W0718: Catching too general exception Exception (broad-exception-caught)
auction_system.py:53:0: C0116: Missing function or method docstring (missing-function-docstring)
auction_system.py:53:0: C0103: Function name "topBidders" doesn't conform to snake_case naming style (invalid-name)
auction_system.py:58:8: R1724: Unnecessary "else" after "continue", remove the "else" and de-indent the code inside
  it (no-else-continue)
auction_system.py:64:0: C0116: Missing function or method docstring (missing-function-docstring)
auction_system.py:64:0: C0103: Function name "renderListing" doesn't conform to snake_case naming style (invalid-name)
auction_system.py:69:0: C0116: Missing function or method docstring (missing-function-docstring)
auction_system.py:69:0: C0103: Function name "bidderBadge" doesn't conform to snake_case naming style (invalid-name)
auction_system.py:72:0: C0116: Missing function or method docstring (missing-function-docstring)
auction_system.py:72:0: C0103: Function name "chargeWinner" doesn't conform to snake_case naming style (invalid-name)
auction_system.py:77:0: C0103: Function name "exportBids" doesn't conform to snake_case naming style (invalid-name)
auction_system.py:83:0: C0116: Missing function or method docstring (missing-function-docstring)
auction_system.py:83:0: C0103: Function name "secondsLeft" doesn't conform to snake_case naming style (invalid-name)
auction_system.py:88:0: C0116: Missing function or method docstring (missing-function-docstring)
auction_system.py:3:0: C0411: standard import "time" should be placed before third party imports "jinja2.Environment",
   "markupsafe.Markup" (wrong-import-order)
auction_system.py:4:0: C0411: standard import "json" should be placed before third party imports "jinja2.Environment",
   "markupsafe.Markup" (wrong-import-order)
auction_system.py:5:0: C0411: standard import "xmlrpc.client" should be placed before third party imports "jinja2.
  Environment", "markupsafe.Markup" (wrong-import-order)
auction_system.py:6:0: C0411: standard import "datetime.datetime" should be placed before third party imports "jinja2.
  Environment", "markupsafe.Markup" (wrong-import-order)

-----------------------------------
Your code has been rated at 6.09/10
```

pylint_report_after.txt

```
-------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 9.89/10, +0.11)
```

bandit_report_before.txt

```
Run started:2026-10-30 16:55:10.734347+00:00

Test results:
>> Issue: [B411:blacklist] Using xmlrpc.client to parse untrusted XML data is known to be vulnerable to XML attacks.
  Use defusedxml.xmlrpc.monkey_patch() function to monkey-patch xmlrpclib and mitigate XML vulnerabilities.
   Severity: High   Confidence: High
   CWE: CWE-20 (https://cwe.mitre.org/data/definitions/20.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b411-import-xmlrpclib
```

```
   Location: .\auction_system.py:5:0
4   import json
5   import xmlrpc.client
6   from datetime import datetime

--------------------------------------------------
>> Issue: [B701:jinja2_autoescape_false] Using jinja2 templates with autoescape=False is dangerous and can lead to
  XSS. Use autoescape=True or use the select_autoescape function to mitigate XSS vulnerabilities.
   Severity: High   Confidence: High
   CWE: CWE-94 (https://cwe.mitre.org/data/definitions/94.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b701_jinja2_autoescape_false.html
   Location: .\auction_system.py:65:10
64  def renderListing(item_id):
65      env = Environment(autoescape=False)
66      template = env.from_string("<h2>{{ a.title }}</h2><p>Current bid: Rs.{{ a.highest }}</p>")

--------------------------------------------------
>> Issue: [B704:markupsafe_markup_xss] Potential XSS with ``markupsafe.Markup`` detected. Do not use ``Markup`` on
  untrusted data.
   Severity: Medium   Confidence: High
   CWE: CWE-79 (https://cwe.mitre.org/data/definitions/79.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b704_markupsafe_markup_xss.html
   Location: .\auction_system.py:70:11
69  def bidderBadge(name):
70      return Markup("<span class='bidder'>" + name + "</span>")
71

--------------------------------------------------

Code scanned:
    Total lines of code: 90
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 1
        High: 2
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 3
Files skipped (0):
```

bandit_report_after.txt

```
Run started:2026-10-30 17:28:54.897090+00:00

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 132
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
Files skipped (0):
```

flake8_report_before.txt

```
auction_system.py:11:1: E302 expected 2 blank lines, found 1
auction_system.py:43:1: E302 expected 2 blank lines, found 1
auction_system.py:47:1: E302 expected 2 blank lines, found 1
auction_system.py:53:1: E302 expected 2 blank lines, found 1
```

```
auction_system.py:64:1: E302 expected 2 blank lines, found 1
auction_system.py:66:80: E501 line too long (94 > 79 characters)
auction_system.py:69:1: E302 expected 2 blank lines, found 1
auction_system.py:72:1: E302 expected 2 blank lines, found 1
auction_system.py:77:1: E302 expected 2 blank lines, found 1
auction_system.py:83:1: E302 expected 2 blank lines, found 1
auction_system.py:88:1: E302 expected 2 blank lines, found 1
auction_system.py:89:80: E501 line too long (83 > 79 characters)
auction_system.py:90:80: E501 line too long (109 > 79 characters)
auction_system.py:93:80: E501 line too long (87 > 79 characters)
auction_system.py:94:80: E501 line too long (81 > 79 characters)
auction_system.py:95:80: E501 line too long (90 > 79 characters)
auction_system.py:96:80: E501 line too long (94 > 79 characters)
auction_system.py:101:80: E501 line too long (88 > 79 characters)
auction_system.py:107:1: E305 expected 2 blank lines after class or function definition, found 1
```

flake8_report_after.txt

```
(empty file: Flake8 reported no issues)
```

5. Results After Fixing the Issues

```
(venv) PS D:\PES_5TH_SEM\SE\LAB\LAB_5\SE-Labs-PES1UG25CS824\LAB5> type
 pylint_report_after.txt

 --------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 9.89/10, +0.11)

(venv) PS D:\PES_5TH_SEM\SE\LAB\LAB_5\SE-Labs-PES1UG25CS824\LAB5>
```

Figure 5 – Pylint after fixing: 10.00/10.

```
Terminal - LAB5

(venv) LAB5> python -m bandit -r auction_system.py

Run started:2026-10-30 17:28:54.897090+00:00

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 132
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
Files skipped (0):
```

Figure 6 – Bandit after fixing: No issues identified.

```
Terminal - LAB5

(venv) LAB5> python -m flake8 auction_system.py

(no output: no issues found)
```

Figure 7 – Flake8 after fixing: no output, i.e. no violations.

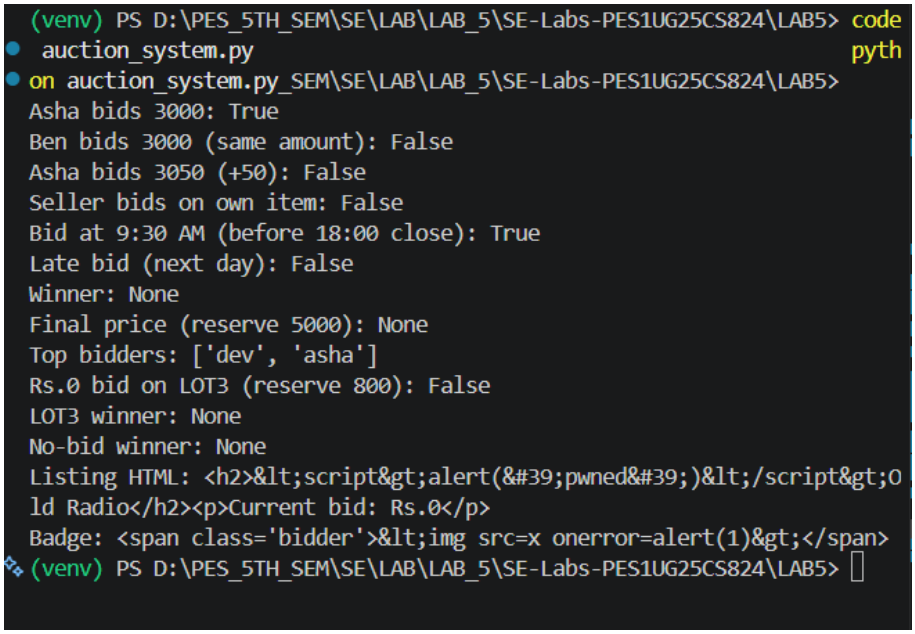


Figure 8 – Program output after fixing: invalid bids are rejected, the 9:30 bid is accepted, no winner below the reserve, and the HTML is escaped.

5.1 Key code changes

Before	After
`import xmlrpc.client`	`import defusedxml.xmlrpc` `defusedxml.xmlrpc.monkey_patch()`
`Environment(autoescape=False)`	`Environment(autoescape=True)`
`Markup("<span class='bidder'>" + name + "</span>")`	`Markup("<span class='bidder'>{}</span>").format(name)`
`if at > self.ends_at:`	`if parse_time(at) > self.ends_at:`
`if amount >= self.highest:`	`if bidder == self.seller or amount <= 0: return False` `if self.bids and amount < self.highest + MIN_INCREMENT: return False`
`raise Exception("Auction closed")` `except Exception:`	`raise AuctionClosedError("Auction closed")` `except (KeyError, AuctionClosedError):`
`return self.leader`	`if not self.bids or self.highest < self.reserve_price: return None`
`main()`	`if __name__ == "__main__":` `  main()`

5.2 Commit history (LAB5)

- Add original Lab 5 auction_system.py
- Add static analysis reports before fixes
- **Fix Bandit security issues (B411, B701, B704)**
- **Fix Pylint issues and logic bugs**
- **Fix Flake8 style issues and document findings**
- Add Lab 5 reflection; add final newline (Pylint 10.00, Flake8 clean); add screenshots

6. Reflection

Q1. Which issues were the easiest to fix, and which were the hardest? Why?

The easiest were the Flake8 style warnings (E302/E305 blank lines and E501 long lines) and Bandit's B701: they were purely mechanical – adding blank lines, wrapping long lines and changing autoescape=False to autoescape=True in one place. Pylint's naming (camelCase to snake_case) and missing-docstring warnings were also simple, though every call to the renamed functions had to be updated.

The hardest was B411 (insecure xmlrpc.client). Adding defusedxml.xmlrpc.monkey_patch() alone did not clear the warning, because Bandit flags the import itself, so the import and the ServerProxy had to be switched to defusedxml.xmlrpc. The logic bugs were harder still, because no tool reported them. They were found only by reading the program output: times compared as strings (a valid 9:30 bid was rejected), the reserve price ignored, equal and +50 bids accepted although MIN_INCREMENT was defined, a seller bidding on their own item, and a Rs.0 bid being accepted.

Q2. Did the static analysis tools report any false positives? If so, describe one example.

Partly. Bandit's B411 is arguably a false positive here: the payment gateway URL is a fixed, trusted server, yet Bandit flags any use of xmlrpc.client regardless of context. It was still fixed, because defusedxml costs little and protects against a compromised or spoofed server. Some Pylint messages were style preferences rather than defects, e.g. R0902 (too many instance attributes, 8/7) and missing docstrings – the code worked despite them. The tools also had false negatives: none of them detected the logic bugs, which were the most serious problems in the program.

Q3. How would you integrate static analysis tools into your actual software development workflow?

- **Local development:** run Flake8 and Pylint inside the editor (VS Code extensions) so issues appear while typing, and use a pre-commit hook that runs Flake8, Pylint and Bandit before each commit.
- **Continuous Integration:** a GitHub Actions workflow runs all three tools on every push and pull request, and fails the build if Bandit reports a High-severity issue, Flake8 reports any error, or the Pylint score drops below 9.5.
- **Shared configuration:** keep settings (line length, disabled checks) in setup.cfg or .pylintrc so the whole team uses the same rules.
- **Complementary practices:** because the tools cannot catch logic errors, combine them with unit tests and code review.

Q4. What tangible improvements did you observe in the code quality, readability, or potential robustness after applying the fixes?

- **Scores:** Pylint went from 6.09/10 to 10.00/10, Bandit from 3 issues (2 High, 1 Medium) to none, and Flake8 from 19 warnings to 0.
- **Security:** auction titles and bidder names are HTML-escaped, so the <script> and <img onerror> XSS payloads no longer run, and XML-RPC responses are parsed safely with

defusedxml.

- **Robustness:** a custom AuctionClosedError replaced the generic raise/except Exception, so unrelated errors are no longer silently hidden; highest and leader are properties, so leader can never be undefined.
- **Correctness:** bids must beat the highest by MIN_INCREMENT, sellers cannot bid on their own items, zero bids are rejected, times are compared as real datetimes, and no winner or price is returned when the reserve is not met.
- **Readability:** snake_case names, docstrings on every function, grouped imports and an if __name__ == "__main__" guard make the code easier to read and safe to import.

Conclusion. Static analysis quickly found the style, quality and security problems in the program and gave a measurable way to confirm the fixes. It does not replace testing: the most serious defects here were logic errors that only careful reading and running of the program revealed.